

Université Marie et Louis Pasteur

Master 2 Internet of Things

A Border-Tracing Algorithm on Blinky Blocks in VisibleSim

Author:
Muhammad Shiraz

Submitted to:
Dr. Jad Bassil

October 18, 2026

Abstract

This report details the design, implementation, and evaluation of a distributed border-tracing algorithm tailored for 2D lattice-based modular robotic systems (Blinky Blocks) within the VisibleSim framework. Operating under decentralized neighborhood communication constraints, the algorithm successfully identifies both external boundaries and internal holes within complex configurations. To address memory limitations inherent to resource-constrained embedded nodes, a custom collinear compression and bit-packing mechanism is proposed and implemented. This compressed representation enables decentralized ray-casting validation across individual modules. Extensive simulation results across multiple structured layouts confirm the precision, communication efficiency, and optimal scaling of the developed solution.

Contents

1 Problem Statement and Objectives **3**

2 System Assumptions **3**

3 Algorithm Description **3**
3.1 Initiator Condition and Routing Logic 3

4 Collinear Module Optimization **4**
4.1 Proposed Solution and Bit-Packing Architecture 4

5 Complexity Analysis **4**
5.1 Time Complexity 4
5.2 Communication Complexity 5
5.3 Memory Complexity 5

6 Validation and Simulation Results **5**

7 Conclusion **5**

1 Problem Statement and Objectives

Modular robotic systems frequently operate under decentralized conditions where individual computational units lack a global view of their environment.

- **The Problem:** How to compute and store a memory-efficient representation of a configuration's global borders when each individual module possesses only a limited, neighbor-to-neighbor perspective, while correctly accounting for complex structures containing both external perimeters and internal cavities (holes).
- **The Goal:** Design, implement, and evaluate a distributed message-passing algorithm where specific initiator modules autonomously trace the perimeter of the structure. Once boundaries are traced, the algorithm must compress this geometric data into a minimal footprint so it can be broadcast across the network, allowing any module to independently verify its position relative to the global structure via local ray-casting.

2 System Assumptions

The implementation relies on several core architectural and operational assumptions regarding the modular robot platform:

- **Lattice Topology:** Modules are positioned on the cells of a regular 2D square lattice, where each module possesses precise local knowledge of its coordinates and orientation.
- **Localized Communication:** Communication is strictly restricted to neighbor-to-neighbor message passing via physical connectors, allowing receiving nodes to precisely identify the relative direction of incoming messages.
- **Decentralized Processing:** All computations are performed locally based solely on information obtained through local neighborhood interactions.
- **Static Configuration:** The network topology remains fixed and fully connected throughout the execution of the algorithm, with no modules dynamically joining or leaving.

3 Algorithm Description

Because the robotic units reside on a 2D square grid, each cell features four direct neighbors connected along its orthogonal borders. A module is designated as a border element if it maintains at least one empty direct neighboring cell. The algorithm systematically handles two distinct classifications of boundaries:

1. **External Borders:** The primary enclosing perimeter of the modular assembly.
2. **Internal Borders:** Bounding loops that isolate internal empty spaces, representing structural holes.

3.1 Initiator Condition and Routing Logic

The distributed search procedure is triggered automatically by modules satisfying the local structural predicate:

$$\text{initiator}(x, y) = \text{empty}(x, y-1) \wedge \big[\text{empty}(x-1, y) \vee \neg\text{empty}(x+1, y-1)\big] \tag{1}$$

This condition guarantees that every unique boundary component (whether internal or external) contains at least one designated initiator. Initiators generate a `BORDER_SEARCH_MSG` containing

the initiator's coordinate signature and route it along the perimeter using a strict right-hand wall-following rule determined by local connectivity checks.

To resolve race conditions caused by multiple initiators spawning along the same continuous border, a deterministic priority rule is enforced. A receiving module compares incoming message initiator coordinates using a lexicographical ordering function:

$$(x_1, y_1) < (x_2, y_2) \iff (x_1 < x_2) \vee \big((x_1 = x_2) \wedge (y_1 < y_2)\big) \tag{2}$$

This ensures that only the initiator possessing the minimum coordinate pair successfully completes a full circuitous traversal. Upon returning to the minimum initiator, the border tracing concludes, and the data is broadcast network-wide via `BORDER_DONE_MSG` packets.

4 Collinear Module Optimization

Storing every raw coordinate along extended straight boundaries introduces excessive memory overhead. To mitigate this, a custom optimization pipeline is integrated to exploit collinearity.

4.1 Proposed Solution and Bit-Packing Architecture

The optimization proceeds through three structured phases:

1. **Corner Extraction:** The raw sequence of boundary modules is parsed to detect directional inflections. Intermediate modules sharing identical entry and exit movement vectors are filtered out, reducing the path description strictly to its geometric vertices.
2. **Vertical Edge Decomposition:** The extracted polygon vertices are converted into a canonical set of vertical bounding segments defined by a column index k and vertical span bounds $(y_{\min}, y_{\max})$.
3. **Bit-Level Compression:** Rather than utilizing standard 32-bit integers, the edge attributes are dynamically packed into a compact byte stream by computing minimum base offsets and utilizing variable-width bit fields.

Representation Type	Data Granularity	Storage Efficiency	Memory Scale
Raw Coordinate List	Full (x, y) per module	Lowest	$O(P)$
Collinear Corner Reduction	Vertex-only coordinates	Moderate	$O(C)$
Bit-Packed Vertical Edges	Compressed offset stream	Highest	$O(E_{\text{vert}})$

Table 1: Comparison of border representation memory tiers.

5 Complexity Analysis

The formal analytical complexity of the distributed algorithm is evaluated across temporal, communicational, and spatial metrics:

5.1 Time Complexity

Tracing a single border requires visiting each perimeter module sequentially, yielding an operational bound of $O(P)$, where P represents the perimeter module count. The subsequent broadcast phase distributes the payload across the network diameter D. Consequently, the total worst-case time complexity is expressed as:

$$T_{\text{total}} = O(P + D) \tag{3}$$

5.2 Communication Complexity

The perimeter traversal generates exactly P localized point-to-point message transmissions. The dissemination phase floods the network to ensure global storage across all N modules, resulting in:

$$C_{\text{total}} = O(P + N) \tag{4}$$

5.3 Memory Complexity

Without optimization, local module storage scales linearly with perimeter size $O(P)$. By enforcing collinear vertex reduction and bit-packing, local storage consumption is bounded by the number of geometric corners C, where $C \ll P$:

$$M_{\text{local}} = O(C) \tag{5}$$

6 Validation and Simulation Results

The implementation was rigorously tested within the VisibleSim framework using Blinky Blocks modules across three diverse geometric topologies: a symmetric Space Invader, an intricate Game Controller, and a structured Castle layout. Ray-casting verification confirmed 100

Configuration	**Total Modules (N**	**Perimeter Size (P)**	**Total Messages**	**Sim Time (ms)**
Space Invader	57	56	573	74.57
Castle	113	96	840	100.36
Game Controller	87	80	1195	70.26

Table 2: Quantitative performance metrics across validated XML test benches.

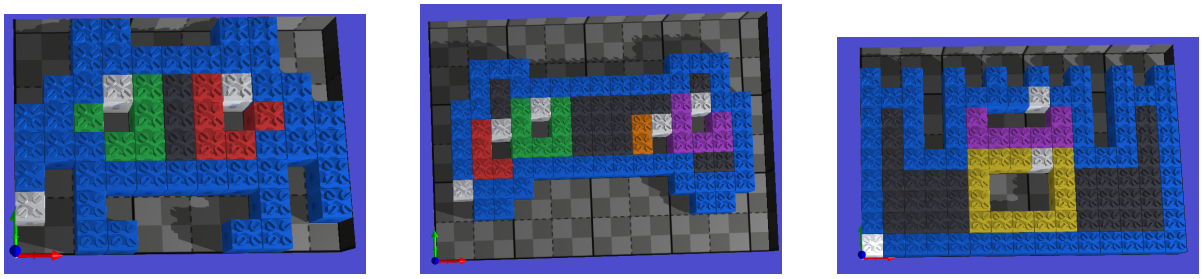


Figure 1: Visual verification of distributed border identification. Left to Right: Space Invader, Game Controller, and Castle configurations.

7 Conclusion

This project successfully demonstrated the feasibility and robustness of decentralized boundary recognition in modular robotic systems. By implementing a robust neighbor-to-neighbor message-passing protocol, Blinky Blocks successfully coordinated to identify complex external contours and internal cavities without centralized oversight. Furthermore, the introduction of the custom collinear vertex reduction and bit-packing optimization successfully minimized memory overhead, reducing storage complexity from linear perimeter dependency to an efficient corner-based scale. Ray-casting validation confirmed that individual nodes can accurately determine spatial inclusion locally. Future work will focus on dynamic topology adaptation and handling asynchronous communication faults in reconfigurable modular robotics.